DEMO 2 · 10 / 21

Computational Storage

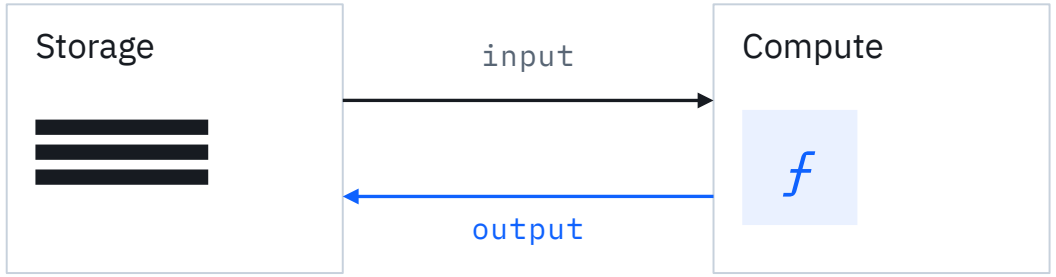
A working prototype and its separate-compute baseline

Aakash, Harshada, Ivan, Jeehan, Mason

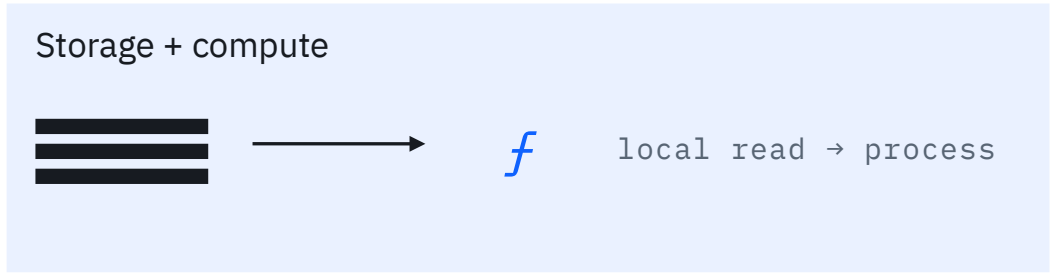
Why computational storage?

Recap from Demo 1: preprocessing moves data twice

Separate



Combined



DROID, RLDS → LEROBOT

1.7 TB → ≈ 400 GB

input and output sizes in Hugging Face's porting guide

GOAL IN THE PROPOSAL

≈ −70%

network bytes,
with runtime no worse than the baseline

Demo 2 milestone

As committed in the design proposal: all four parts are done

DONE

Single-node Ceph cluster

EXPERIMENT 01

MicroCeph (Ceph 19.2): 1 mon, 1 mgr, 1 mds, 3 OSDs; CephFS mounted on both nodes

DONE

Load → convert → write-back

EXPERIMENTS 02–05

Real DROID 1.0.1 shards, Hugging Face's converter unmodified, verified output

DONE

Background conversion service

EXPERIMENT 06

Converts newly loaded data with no manual step

DONE

Separate-compute baseline

EXPERIMENTS 03, 07

797 MB across the network and 304 s per shard, reproduced in a second run

Also measured: the same jobs run on the storage node, compared against the baseline (results on slides 8–9)

What changed since Demo 1

From the proposal to a running system

	DEMO 1: PROPOSED	DEMO 2: BUILT
Storage	Ceph on Kubernetes with Rook	Single-node MicroCeph + CephFS; Rook arrives with multiple nodes in Demo 3, as planned
Interface	Function, inputs and dependencies	A job is a function, its declared input files and an output directory; Demo 3's placement reads the input list
Workers	Containers beside storage, with limits	One pinned image on both nodes: 7 CPUs, 8 GiB, no network inside
Workload	RLDS → LeRobot with Hugging Face's converter	port_droid.py v0.6.1, unmodified, on real DROID 1.0.1 shards
Measurement	Cross-host bytes and end-to-end time	NIC counters, cold caches, and a byte-level comparison of the outputs

Testbed

Two EC2 m6i.2xlarge in one subnet, with the same worker image and limits on both nodes

Compute node

BASELINE: SEPARATE MODE

CephFS client

/mnt/cephfs, same paths as storage

Worker container

port_droid.py · 7 CPUs · 8 GiB · no network

Local scratch for the converter's temp files

runs the experiments, drops caches, reads NIC counters

NETWORK

separate mode: shard read, output written back

cross-host bytes = the storage node's NIC rx + tx

Storage node

MICROCEPH · CEPH 19.2

Ceph + CephFS

mon · mgr · mds · 3 OSDs · 3× replication

Worker container

storage mode: same image, same limits

cstore-service: converts newly loaded data

DROID shards loaded from Google Cloud Storage, MD5-checked

How one job runs

One DROID shard in, one LeRobot dataset out (times measured on shard 2)

01

Job spec

Function, declared input files, output directory

`jobs.py`

→

02

Convert

Reads the shard from CephFS, writes to local scratch

`301 s`

→

03

Publish

Copy and fsync onto CephFS, then one atomic rename

`0.8 s · 141.5 MB`

→

04

Verify

LeRobot's loader; frame counts across data and 3 videos

`39 / 39 episodes`

DESIGN DECISIONS

Reuse the converter unmodified

Hugging Face's port_droid.py, verified by checksum before every run, so where it runs is the only thing that changes.

Same container in both modes

Same image, CPU and memory limits, and no network inside: every byte goes through CephFS, where we count it.

Publish atomically

The service and other readers never see a half-written dataset, and a dataset is either complete or absent.

Background conversion service

Newly loaded data is converted with no manual step (demo video)

1. **Shard loaded**
   bin/cstore load-droid writes it to CephFS
2. **Service notices it**
   no LeRobot output yet → a job in queue/pending/
3. **Claimed and run**
   atomic rename into running/, then a worker runs it
4. **Done and verified**
   done/, a result record, VERIFY OK

```
SERVICE LOG AND VERIFICATION · AWS RUN 2, 2026-10-18

18:54:53  queued    svc-shard-00002
          new shard 2 has no converted output

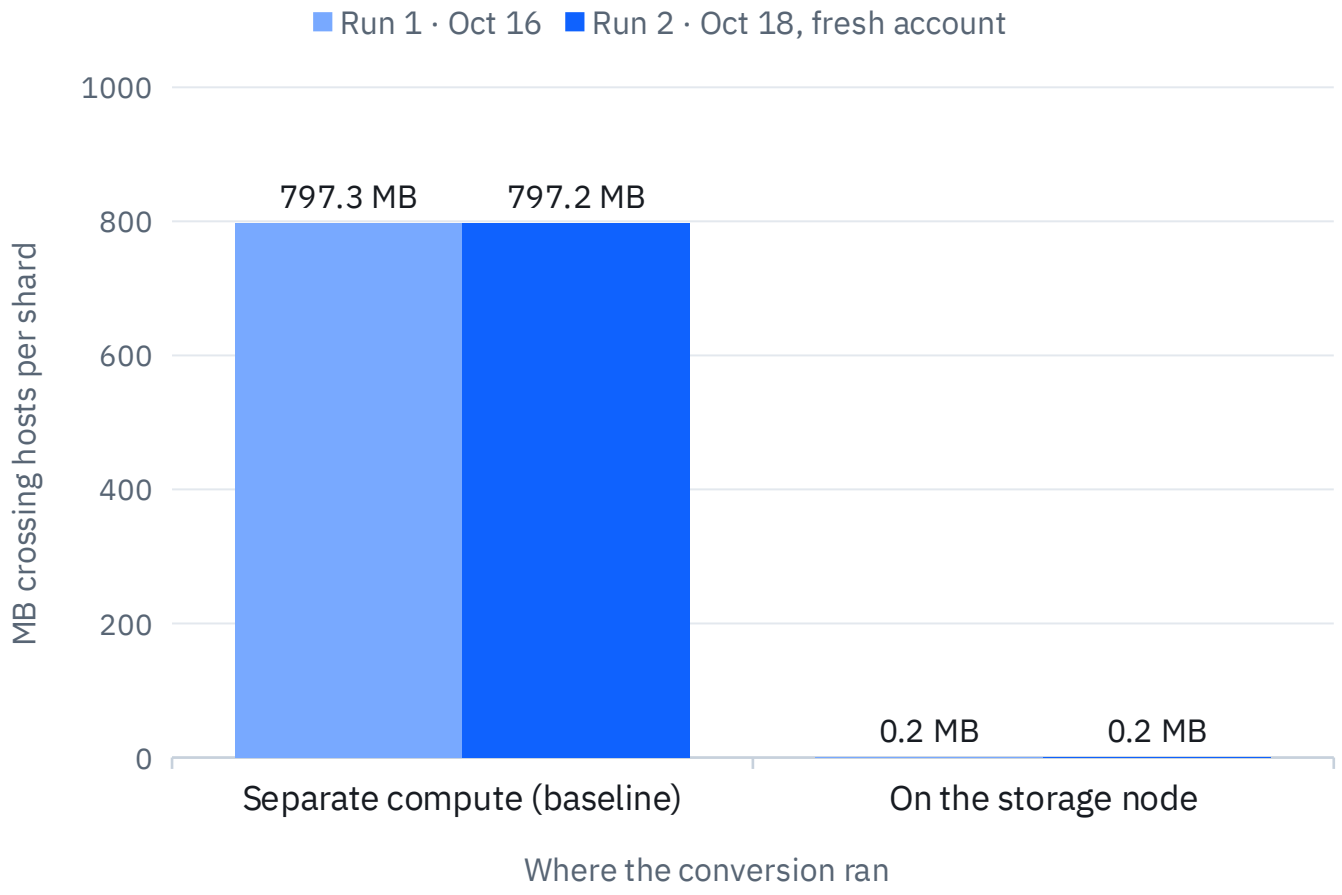
18:54:54  running   droid_rlds_to_lerobot

18:59:46  finished wall 292.5 s · 141.5 MB
          VERIFY OK · 39 episodes · 9,829 frames
```

292 s from queued to done, with no manual step

Result: network traffic

Shard 2, two independent runs; same image, limits and cold caches in both modes



−99.97%

cross-host traffic vs. the baseline

Baseline: 797.3 and 797.2 MB, which is 1.01 × (input + output): the shard is read once and its output written back once

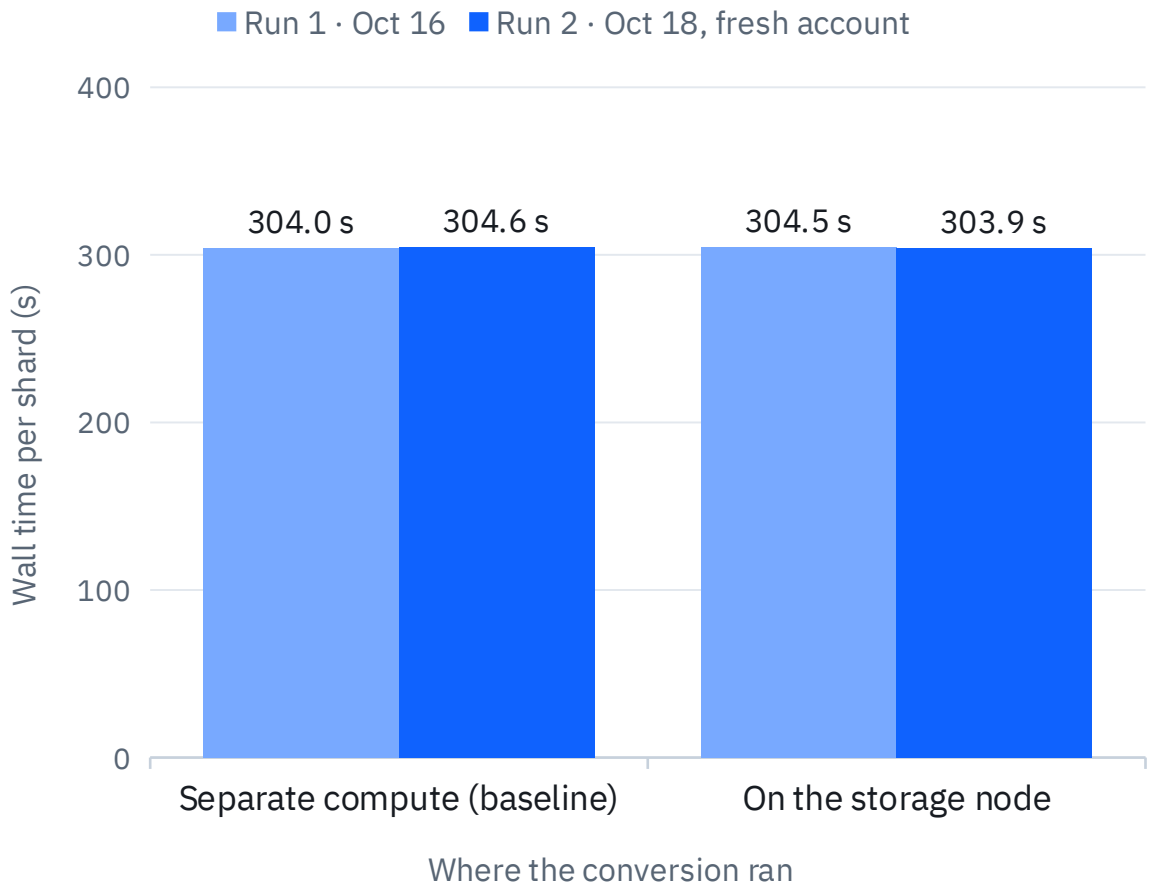
Storage node: 0.2 MB in both runs, ssh and CephFS session traffic

```
PROPOSAL GOAL ≈ −70%

RUN 1  demo2-20261016-210744
RUN 2  demo2-20261018-184414
```

Result: runtime and correctness

No slowdown, and the same dataset from both modes, in both runs



1.00

runtime ratio in both runs, storage node / baseline

Which mode was faster flipped between runs, so the gap is noise.
CPU-bound: 301 s converting vs. under 2 s to publish

Identical

output from both modes, byte for byte, in each run

stats.json compared as canonical JSON (its key order varies)

39 / 39

episodes · 9,829 frames · VERIFY OK

first and last frames decoded by LeRobot's loader, all 3 cameras

Limitations and next steps

What this result does not show yet, and what Demo 3 adds

LIMITATIONS

- One storage node: every input is local by construction, so this is the best case
- This converter is CPU-bound, so locality saves bytes but not time
- Loop-file OSDs share one disk: storage throughput is not representative

DEMO 3 · 11 / 16

- Several storage nodes on Kubernetes with Rook
- Jobs submitted through a serverless API
- Dependency-aware placement: run where the inputs are, fetch the fewest missing bytes
- This baseline rerun on the multi-node setup

≈ 2.1 TB would cross the network to convert all of DROID's 1.7 TB RLDS release with separate compute (estimated from shard 2, not measured)